Sistema Distribuido de Consulta Inteligente de Bibliotecas

Alejandro Mier Cadrazco

Juan Felipe Villaquiran

Gabriel Esteban Badillo

Universidad Manuela Beltran

Ingeniería de Software

Jamilton Fernando Benavides

Sistemas Transaccionales

5 de noviembre de 2026

Tabla de contenido

1. Abstract **5**

2. Introducción **6**

3. Justificación **6**

4. Problemática **7**

5. Necesidad **7**

6. Pregunta problema **7**

7. Objetivo central **7**

8. Objetivos específicos **8**

9. Marco conceptual **9**

9.1. Conceptos fundamentales del proyecto 9

9.2. Conceptos de sistemas transaccionales 10

9.3. Conceptos tecnológicos 11

9.4. Aplicación de los conceptos al proyecto 12

10. Estado del arte **14**

10.1. Sistemas integrados de gestión bibliotecaria 14

10.2. Búsqueda federada y protocolos de interoperabilidad 14

10.3. Consultas en lenguaje natural sobre bases de datos 15

10.4. Posición del prototipo frente a lo existente 16

11. Marco contextual **17**

11.1. Contexto del proyecto 17

11.2. Problemática identificada 17

11.3. Usuarios y beneficiarios 17

11.4. Necesidad que se pretende atender 18

11.5. Descripción del prototipo 18

11.6. Alcance del proyecto 19

12. Requerimientos del sistema **21**

12.1. Requerimientos funcionales 21
12.2. Requerimientos no funcionales 21
12.3. Detalle de los requerimientos no funcionales 21
12.4. Trazabilidad entre requerimientos, casos de uso y modelo de datos 26
13. Arquitectura de software 27
13.1. Visión general 27
13.2. Operación distribuida: tiempos de espera y errores parciales 28
14. Diagrama de casos de uso UML 31
15. Diagrama de clases UML 37
16. Modelo relacional 38
17. Marco integrador: servicio de Inteligencia Artificial 39
17.1. Servicio de Inteligencia Artificial propuesto 39
17.2. Función dentro del proyecto 39
17.3. Integración con el sistema 40
17.4. Resultado esperado 41
17.5. Límites de esta propuesta 41
18. Red neuronal para la interpretación de consultas 42
18.1. Qué problema resuelve la red y por qué hace falta 42
18.2. Formulación de la tarea 42
18.3. Arquitectura propuesta 43
18.4. Conjunto de datos 45
18.5. Entrenamiento y evaluación 46
18.6. Integración con el resto del sistema 47
18.7. Límites y riesgos 47
19. Evidencias de pruebas transaccionales 49
19.1. Alcance y método 49
19.2. Resultados obtenidos 49
19.3. Los dos casos que sostienen el diseño 50

19.4. Qué queda sin cubrir 51

20. Gestión del proyecto 53

20.1. Enfoque de trabajo 53

20.2. Roles y responsabilidades 53

20.3. Cronograma y hitos alcanzados 54

20.4. Gestión de la configuración 54

20.5. Riesgos identificados 55

20.6. Estado actual y plan de continuidad 56

21. Conclusiones 58

22. Referencias Bibliográficas 60

1. Abstract

The project proposes a distributed desktop application for managing and intelligently consulting books in three independent libraries. The system will be developed in Python and will use gRPC/ RPC for communication between library nodes, SQL Server for data management, and an Artificial Intelligence module for natural-language queries. The solution will allow users to consult book availability across the three libraries, as well as manage users, books, loans, and returns. Transactional mechanisms based on ACID properties will be applied to protect data integrity. The project also establishes a software architecture that integrates the desktop application, AI module, distributed communication, and independent databases, providing a practical solution for efficient and reliable library resource management.

2. Introducción

El proyecto “Sistema Distribuido de Consulta Inteligente de Libros en Tres Bibliotecas” propone una aplicación de escritorio desarrollada en Python para gestionar y consultar recursos bibliográficos distribuidos en tres bibliotecas independientes.

Actualmente, cuando la información se encuentra separada entre diferentes bibliotecas, consultar la disponibilidad de un libro puede requerir búsquedas individuales en cada ubicación. Para mejorar este proceso, el sistema integrará las tres bibliotecas mediante una comunicación distribuida con gRPC/RPC.

Además, contará con un módulo de Inteligencia Artificial que permitirá realizar preguntas en lenguaje natural, obteniendo respuestas basadas en los datos reales de las bases de datos.

Desde el punto de vista de la Arquitectura de Software, el proyecto plantea una estructura compuesta por una aplicación de escritorio, un módulo de IA, servicios de comunicación y tres nodos con bases de datos independientes.

3. Justificación

El proyecto es importante porque busca facilitar el acceso a la información bibliográfica cuando ésta se encuentra distribuida entre diferentes bibliotecas con locaciones distintas y muchas veces ni siquiera cercanas. El sistema permitirá consultar desde una misma aplicación dónde se encuentra disponible un libro y cuántos ejemplares existen.

También beneficiará a los administradores y bibliotecarios al facilitar la gestión de libros, usuarios, préstamos y devoluciones. La incorporación de Inteligencia Artificial permitirá realizar consultas de forma más sencilla mediante lenguaje natural.

Desde el punto de vista académico, el proyecto permite aplicar conocimientos de Arquitectura de Software, sistemas distribuidos, bases de datos, transacciones, comunicación entre servicios e Inteligencia Artificial en una solución integrada.

4. Problemática

La información de las tres bibliotecas se encuentra distribuida en diferentes sistemas y ubicaciones. Esto puede dificultar la consulta de libros y disponibilidad, ya que un usuario podría tener que revisar cada biblioteca por separado.

Además, las operaciones de préstamos y devoluciones requieren mantener información consistente. Un error durante estas operaciones podría generar diferencias entre los registros de préstamos y la cantidad real de libros disponibles.

También existe la dificultad de proporcionar una forma sencilla de consultar información distribuida, especialmente para usuarios que prefieren realizar preguntas en lenguaje natural.

5. Necesidad

Se necesita un sistema que permita consultar y gestionar la información de las tres bibliotecas desde un único punto de interacción, manteniendo cada biblioteca como un nodo independiente.

También se requiere una herramienta que facilite las consultas mediante lenguaje natural y garantice la integridad de operaciones como préstamos y devoluciones mediante transacciones de base de datos.

6. Pregunta problema

¿Cómo diseñar un sistema distribuido de escritorio que permita gestionar y consultar los recursos de tres bibliotecas, garantizando la integridad de las operaciones y facilitando la búsqueda de libros mediante lenguaje natural?

7. Objetivo central

Diseñar un sistema distribuido de escritorio que permita gestionar y consultar los recursos bibliográficos de tres bibliotecas independientes, integrando comunicación mediante gRPC/RPC, bases de datos SQL Server e Inteligencia Artificial para facilitar las consultas en lenguaje natural, garantizando la integridad de las operaciones de préstamos y devoluciones.

8. Objetivos específicos

- Implementar la gestión de libros, usuarios, préstamos y devoluciones, manteniendo actualizada la información de los ejemplares y su disponibilidad en cada biblioteca.
- Integrar un módulo de Inteligencia Artificial que permita realizar consultas en lenguaje natural y obtener información sobre la ubicación y disponibilidad de los libros en las tres bibliotecas.
- Aplicar mecanismos transaccionales basados en propiedades ACID para garantizar la integridad y consistencia de la información durante las operaciones de préstamos y devoluciones.

9. Marco conceptual

Antes de entrar en los requerimientos conviene dejar claros los conceptos que sostienen el diseño. Cada uno se explica junto con la parte del prototipo donde aparece, porque una definición suelta no dice mucho.

9.1. Conceptos fundamentales del proyecto

Catálogo bibliográfico, título y ejemplar

Una biblioteca no maneja libros, maneja títulos y ejemplares. El título es la obra identificada por su ISBN, autor y editorial. El ejemplar es cada copia física que la sede tiene de ese título. La diferencia parece de palabras, pero no lo es: un título puede estar en el catálogo y aun así no haber ninguna copia libre, porque todas están prestadas. Por eso, cuando el sistema habla de disponibilidad, se refiere siempre a ejemplares.

Préstamo, devolución y mora

El préstamo es la operación en la que un ejemplar pasa temporalmente a manos de un usuario, con una fecha de vencimiento acordada. La devolución lo cierra y el ejemplar regresa al inventario disponible. Si esa fecha pasa y el libro no vuelve, el préstamo queda en mora. El sistema tiene que darse cuenta de eso, porque de ahí depende que al mismo lector se le preste o no otro libro.

Disponibilidad distribuida

Es el dato de cuántos ejemplares libres de un título hay entre las tres bibliotecas juntas. Ninguna sede lo tiene, ni puede tenerlo por su cuenta. Se arma en el momento de la consulta: se les pregunta a las tres y se suma lo que responden. Todo el proyecto gira alrededor de este dato.

9.2. Conceptos de sistemas transaccionales

Transacción y propiedades ACID

Una transacción agrupa varias operaciones sobre la base de datos y las ejecuta como si fueran una sola. El acrónimo ACID viene de Härder y Reuter (1983) y nombra las cuatro propiedades que el gestor de base de datos tiene que garantizar en cada una:

- **Atomicidad:** o se ejecuta toda la transacción, o no se ejecuta nada. No quedan estados intermedios visibles.
- **Consistencia:** la base pasa de un estado válido a otro estado válido, respetando todas sus restricciones de integridad.
- **Aislamiento:** varias transacciones que corren al mismo tiempo dan el mismo resultado que si se hubieran ejecutado una tras otra.
- **Durabilidad:** cuando una transacción se confirma, el cambio queda guardado aunque el sistema falle enseguida.

Procesamiento transaccional en línea

Este sistema es de procesamiento transaccional en línea. Son muchas operaciones cortas, como registrar un préstamo, buscar un título o cerrar una devolución, que varios usuarios hacen al mismo tiempo y que tienen que confirmarse de una vez. Importa más que el dato quede exacto que la cantidad de información procesada.

Concurrencia, bloqueo y nivel de aislamiento

Si dos bibliotecarios intentan prestar al mismo tiempo el último ejemplar de un título, el sistema tiene que evitar que ambos lo consigan. De eso se encarga el control de concurrencia, que bloquea las filas involucradas mientras dura la operación. El nivel de aislamiento define qué tanto se permite que las transacciones se interfieran entre sí. En SQL Server se fija con la instrucción SET TRANSACTION ISOLATION LEVEL, y por defecto se usa read committed, que evita leer datos sin confirmar (Microsoft, 2026b). Sobre ese nivel se pueden agregar indicaciones de bloqueo que reservan una fila para la transacción que está corriendo.

Integridad referencial y restricciones de dominio

Las llaves foráneas impiden que un préstamo apunte a un usuario o a un ejemplar que no existe. Las restricciones de dominio limitan los valores que acepta una columna, como el estado de un ejemplar, de modo que la propia base rechaza cualquier valor que no esté en la lista prevista. Estas dos cosas son las que sostienen la propiedad de consistencia.

9.3. Conceptos tecnológicos

Sistema distribuido

Un sistema distribuido es un conjunto de componentes autónomos que están en equipos distintos, se coordinan entre sí y se le presentan al usuario como un solo sistema (van Steen y Tanenbaum, 2017). Al proyecto le interesan sobre todo dos cosas de esa definición: que cada nodo manda sobre sus propios datos y que el conjunto aguanta que uno de sus componentes deje de responder.

Llamada a procedimiento remoto y gRPC

La llamada a procedimiento remoto permite que una aplicación invoque un procedimiento que se ejecuta en otro equipo como si estuviera en el mismo. gRPC es un marco de código abierto que lleva esa idea a la práctica. El desarrollador define en un archivo los servicios y los mensajes que se intercambian, y a partir de ahí se genera el código del cliente y del servidor (gRPC, 2024a). Esa definición se escribe en Protocol Buffers, que es a la vez un lenguaje para describir interfaces y un formato binario de serialización, así que el contrato entre los servicios queda tipado y ocupa poco (gRPC, 2024b).

Arquitectura en capas y patrón repositorio

La arquitectura en capas divide el software en niveles con responsabilidades separadas, donde cada capa se apoya solo en la que tiene debajo (Bass et al., 2021). El patrón repositorio lleva el acceso a los datos a objetos dedicados, para que las reglas de negocio no tengan sentencias SQL

adentro. Usando los dos juntos, la interfaz gráfica y el módulo de Inteligencia Artificial pueden aprovechar la misma lógica sin repetirla.

Interfaces en lenguaje natural para bases de datos

Una interfaz en lenguaje natural para bases de datos le permite al usuario escribir su consulta en lengua corriente en vez de hacerlo en un lenguaje formal como SQL. Affolter, Stockinger y Bernstein (2019) revisaron varios de estos sistemas y encontraron que suelen mezclar técnicas lingüísticas con aprendizaje automático para pasar de la pregunta a una consulta estructurada. También señalan que la mayor dificultad está en resolver la ambigüedad de lo que escribe el usuario y en delimitar bien el dominio sobre el que se consulta.

9.4. Aplicación de los conceptos al proyecto

La Tabla 1 muestra en qué parte del prototipo se usa cada uno de los conceptos anteriores.

Tabla 1

Correspondencia entre los conceptos del marco teórico y el prototipo

Concepto	Aplicación en el prototipo
Título frente a ejemplar	Se modelan como dos entidades separadas, relacionadas por una llave foránea. La disponibilidad se calcula contando ejemplares en estado disponible, no títulos del catálogo.
Propiedades ACID	El préstamo y la devolución se ejecutan dentro de una única transacción que abarca varias tablas; cualquier error revierte todos los cambios de la operación.
Control de concurrencia	La selección del ejemplar que se va a prestar se realiza bajo indicaciones de bloqueo, de modo que dos préstamos simultáneos del mismo título tomen copias distintas.
Integridad referencial y de dominio	El esquema de cada nodo declara llaves foráneas y restricciones que limitan los estados admisibles de ejemplares y préstamos, y exigen ISBN y documento únicos.
Autonomía de los nodos	Cada biblioteca administra su propia base de datos. No existe replicación entre sedes ni una base central que concentre el inventario.
Tolerancia a fallos parciales	El cliente consulta las tres bibliotecas en paralelo; si una no responde, informa el resultado de las restantes e indica cuál falló.

Concepto	Aplicación en el prototipo
Llamada a procedimiento remoto	Los nodos exponen sus operaciones mediante un contrato gRPC común, de modo que los tres servidores son intercambiables desde la perspectiva del cliente.
Arquitectura en capas	El servidor se divide en cuatro niveles: contrato de servicios, reglas de negocio, repositorios de datos y conexión a la base de datos.
Interfaz en lenguaje natural	El módulo de IA traduce la pregunta del usuario a los criterios de búsqueda que el contrato gRPC ya admite, sin generar SQL directamente.

Nota. Elaboración propia.

10. Estado del arte

Automatizar una biblioteca no es un problema nuevo y hay software maduro que lo resuelve. Lo que sí sigue abierto es qué hacer cuando las bibliotecas son varias, cada una quiere conservar el control de sus datos y aun así se necesita una respuesta única sobre dónde hay un ejemplar libre. Este apartado revisa cómo se ha abordado hasta ahora y en qué punto se ubica el prototipo.

10.1. Sistemas integrados de gestión bibliotecaria

La categoría de software que atiende el trabajo diario de una biblioteca se conoce como sistema integrado de gestión bibliotecaria. Reúne en un solo producto la catalogación, la circulación de ejemplares, las adquisiciones y la gestión de publicaciones periódicas.

El referente de código abierto es Koha, que su propia comunidad describe como el primer paquete libre de automatización bibliotecaria. El trabajo empezó el 6 de septiembre de 1999, lo desarrolló la empresa Katipo Communications junto con el Horowhenua Library Trust de Nueva Zelanda, y el sistema entró en producción el 3 de enero de 2000. Se distribuye bajo la Licencia Pública General de GNU, versión 3 o posterior, y declara soporte para los estándares MARC 21, UNIMARC, Z39.50, SRU, SIP2 y NCIP (Koha Community, s.f.).

Para el problema de este proyecto lo relevante es cómo resuelven estos sistemas el caso de varias sedes. Koha indica que funciona para consorcios de cualquier tamaño y para bibliotecas de una o varias sucursales (Koha Community, s.f.). Ese funcionamiento se apoya en una instalación común: las sucursales son una división lógica dentro de una misma base de datos, no bases de datos separadas. La consecuencia práctica es que las sedes dependen de un servidor compartido y de la conexión permanente hacia él, que es justamente la condición que la organización del caso descrito en el apartado 11 no acepta.

10.2. Búsqueda federada y protocolos de interoperabilidad

La alternativa histórica a concentrar los datos ha sido consultar catálogos ajenos sin moverlos de su sitio. Para eso se creó Z39.50, un protocolo normalizado de consulta y recuperación de información bibliográfica que permite a un cliente interrogar el catálogo de otra institución con

un lenguaje común, y SRU, su reformulación posterior sobre servicios web. Son los protocolos que sostienen el préstamo interbibliotecario y los catálogos colectivos, y siguen vigentes: Koha los declara entre los estándares que implementa (Koha Community, s.f.).

La idea de fondo de estos protocolos es la misma que adopta el prototipo: preguntar en el momento en lugar de copiar los datos por adelantado. La diferencia está en el alcance. Z39.50 y SRU están pensados para consultar, no para operar. Permiten averiguar si una institución tiene un título, pero no registrar un préstamo con garantías transaccionales sobre el inventario de esa institución. El prototipo necesita las dos cosas.

10.3. Consultas en lenguaje natural sobre bases de datos

El segundo frente del proyecto es dejar que el usuario pregunte con sus palabras. Affolter et al. (2019) revisaron veinticuatro interfaces en lenguaje natural para bases de datos y encontraron que la mayoría convierte la pregunta en una consulta estructurada combinando análisis lingüístico con aprendizaje automático, y que las dos dificultades constantes son la ambigüedad del enunciado y la delimitación del dominio.

La línea más activa de los últimos años es la traducción directa de lenguaje natural a SQL. Yu et al. (2018) publicaron Spider, un conjunto de datos anotado a mano con más de diez mil preguntas sobre doscientas bases de datos de ciento treinta y ocho dominios, construido precisamente para medir si un modelo generaliza a esquemas que no vio durante el entrenamiento. El hecho de que haya hecho falta un conjunto de ese tamaño da una idea de la dificultad del problema.

Para este proyecto esa línea se descartó por una razón de diseño, no de capacidad. Un modelo que escribe SQL necesita permisos sobre la base de datos y puede producir consultas incorrectas que nadie revisó. Como el sistema es transaccional y las cifras que ve el usuario tienen que coincidir con el inventario real, el prototipo restringe el papel del modelo a interpretar la pregunta y entregar criterios de búsqueda sobre un contrato ya definido. Esa decisión queda explicada en el apartado 17 y desarrollada en el 18.

10.4. Posición del prototipo frente a lo existente

La Tabla 2 compara las tres aproximaciones revisadas con la que adopta este proyecto, usando como criterios los que resultan críticos para el caso planteado.

Tabla 2

Comparación entre las aproximaciones existentes y el prototipo

Criterio	**Sistema integrado con sedes (tipo Koha)**	**Búsqueda federada (Z39.50 / SRU)**	**Interfaz que genera SQL**	**Prototipo propuesto**
Ubicación de los datos	Una base compartida; las sedes son divisiones lógicas	Cada institución conserva su catálogo	Depende del sistema consultado	Una base independiente por biblioteca
Autonomía de la sede	Baja: depende del servidor común	Alta	No aplica	Alta: cada nodo opera aunque los demás estén caídos
Operaciones transaccionales remotas	Sí, sobre la base compartida	No, solo consulta	Posible, sin garantías	Sí, con ACID en el nodo dueño del ejemplar
Consulta unificada	Inmediata, por ser una sola base	Sí, armada en el momento	Depende de la consulta generada	Sí, armada en el momento y en paralelo
Consulta en lenguaje natural	No es parte del producto	No	Es su objetivo	Sí, limitada a extraer criterios de búsqueda
Riesgo de dato incorrecto	Bajo	Bajo	Alto: el modelo puede escribir una consulta errónea	Bajo: las cifras salen siempre de los nodos

Nota. Elaboración propia a partir de Koha Community (s.f.), Affolter et al. (2019) y Yu et al. (2018).

El prototipo no pretende competir con un sistema integrado maduro en cantidad de funciones. Lo que aporta es una combinación que las opciones revisadas no ofrecen junta: autonomía total de cada base de datos, operación transaccional sobre el nodo que es dueño del ejemplar, consulta unificada construida en el momento de preguntar y una capa de lenguaje natural acotada de modo que no pueda inventar cifras. Esa combinación es la respuesta a la restricción de la que parte el proyecto, que es la negativa de la organización a entregar el control de sus datos a un servidor común.

11. Marco contextual

11.1. Contexto del proyecto

El proyecto se ubica en una red de tres bibliotecas que pertenecen a la misma organización pero funcionan en sedes separadas. Cada sede tiene su propio personal, su propio inventario y su propio equipo con una instancia de Microsoft SQL Server. Entre ellas no hay una red de datos permanente ni un sistema de información compartido, así que se comunican por teléfono o por correo electrónico cuando necesitan algo.

Esta situación es común en instituciones que fueron creciendo sede por sede, donde cada una escogió sus herramientas antes de que existiera una política común. Pasar todos los datos a un servidor único obligaría a las sedes a depender de una conexión permanente y a entregar el control de su información, y eso la organización no lo acepta. El proyecto parte de esa restricción.

11.2. Problemática identificada

Hoy no hay manera de ver en un solo lugar cuántos ejemplares libres de un título hay entre las tres bibliotecas. El usuario que busca un libro tiene que preguntar sede por sede, y el personal solo puede responder por su inventario local, sin poder decirle si en otra sede sí lo tienen.

De ahí se derivan dos problemas más. El primero es que los préstamos se llevan a mano o en hojas de cálculo que no garantizan que los datos queden consistentes. El segundo es que no queda registro de lo que la gente pregunta, y esa información serviría para decidir qué títulos comprar.

11.3. Usuarios y beneficiarios

Tabla 3

Usuarios del sistema, necesidades y beneficios esperados

Usuario	Necesidad	Beneficio esperado
Usuario Estudiante o público general	Saber si un título está disponible y en qué sede, sin desplazarse ni llamar.	Consulta en lenguaje natural que responde con la disponibilidad real de las tres bibliotecas.
Bibliotecario Personal de atención	Registrar préstamos y devoluciones sin errores y conocer el estado del inventario.	Operación guiada, con validaciones automáticas y control de préstamos vencidos.
Administrador Responsable de la red de bibliotecas	Mantener el catálogo y disponer de información consolidada para la toma de decisiones.	Gestión centralizada del catálogo y reportes de inventario y mora por sede.

Nota. Elaboración propia. Los tres perfiles corresponden a los actores identificados en el diagrama de casos de uso del apartado 14.

11.4. Necesidad que se pretende atender

Lo que se necesita es poder ver de forma confiable la disponibilidad de los títulos en las tres bibliotecas, sin que ninguna pierda el control de sus datos, y tener un registro transaccional de préstamos y devoluciones que mantenga el inventario al día. Hay además una necesidad secundaria, que es bajarle la dificultad a la consulta para el usuario, dejándolo preguntar como preguntaría normalmente en lugar de obligarlo a llenar un formulario con campos técnicos.

11.5. Descripción del prototipo

El prototipo es una aplicación de escritorio hecha en Python que trabaja sobre tres servidores autónomos, uno por biblioteca. Cada servidor maneja su propia base de datos en Microsoft SQL

Server y expone el mismo conjunto de operaciones por gRPC. La aplicación cliente les pregunta a los tres al mismo tiempo y muestra la información reunida.

Información que gestiona. El sistema guarda y administra la ficha de cada biblioteca, el catálogo de títulos con sus datos bibliográficos, los ejemplares físicos con su código de inventario y su estado, los usuarios registrados, los préstamos con sus fechas de entrega y vencimiento, las devoluciones y la bitácora de consultas hechas al módulo de Inteligencia Artificial con los resultados que devolvió cada sede.

Funcionalidades principales. Gestión del catálogo y del inventario, registro y consulta de usuarios, ejecución transaccional de préstamos y devoluciones, consulta de disponibilidad por sede y consolidada, consulta en lenguaje natural, generación de reportes de inventario y mora, y verificación del estado de los nodos.

Propósito. Mostrar que un conjunto de bases de datos autónomas puede dar una experiencia de consulta unificada y confiable, manteniendo las garantías transaccionales en cada nodo y aguantando que cualquiera de ellos se caiga temporalmente.

11.6. Alcance del proyecto

Durante esta etapa académica se desarrollará un prototipo funcional que comprende:

- Las tres bases de datos con su esquema completo y sus restricciones de integridad.
- Los servidores de los tres nodos con el conjunto de operaciones descrito en el apartado 12.
- La ejecución transaccional de préstamos y devoluciones con garantías ACID.
- El cliente distribuido que consulta las tres sedes en paralelo y tolera nodos inactivos.
- El módulo de consulta en lenguaje natural, en el alcance descrito en los apartados 17 y 18.
- La aplicación de escritorio para los tres perfiles de usuario.

Quedan fuera del alcance de esta etapa la puesta en producción en las sedes reales, la migración de los inventarios que ya existen, la conexión con catálogos bibliográficos externos, una versión

web o para dispositivos móviles y el cobro de sanciones por mora. Tampoco se incluye la replicación o sincronización automática de datos entre bibliotecas, que se descartó a propósito desde el diseño.

12. Requerimientos del sistema

Los requerimientos van en el formato institucional de la Universidad Manuela Beltrán, que recoge en una sola hoja los funcionales y los no funcionales. La hoja diligenciada está justo después de este apartado.

12.1. Requerimientos funcionales

Los requerimientos funcionales dicen qué debe permitirle hacer el sistema a sus usuarios. Se redactaron con los atributos que pide la norma ISO/IEC/IEEE 29148:2018: identificador único, enunciado verificable y resultado observable (ISO/IEC/IEEE, 2018). Son diez y agrupan las operaciones que se detallan en los casos de uso del apartado 14.

Intervienen los tres actores del diagrama de casos de uso. El administrador se encarga de la configuración general y de los reportes. El bibliotecario lleva la operación diaria del catálogo, los préstamos y las devoluciones. El usuario, estudiante o público general, consulta la disponibilidad de los títulos. Como el formato no tiene columna de actor, en cada requerimiento se indica dentro de la descripción quién lo ejecuta.

12.2. Requerimientos no funcionales

Los no funcionales no dicen qué hace el sistema sino cómo tiene que comportarse. Se definieron tomando como referencia el modelo de calidad del producto de la norma ISO/IEC 25010, que agrupa los atributos en categorías como adecuación funcional, eficiencia de desempeño, fiabilidad, seguridad, mantenibilidad y capacidad de interacción (ISO/IEC, 2023). A cada uno se le puso un criterio de aceptación que se pueda comprobar sobre el prototipo, para que el cumplimiento no quede en una apreciación.

12.3. Detalle de los requerimientos no funcionales

La hoja institucional deja cada requerimiento en una línea, que alcanza para identificarlo pero no para explicar cómo se consigue. En este apartado se desarrolla cada uno por separado: qué atributo de calidad atiende, con qué decisión de diseño se cumple y con qué evidencia se

comprueba. Las referencias a casos PT corresponden a la suite de pruebas descrita en el apartado 19.

RNF-01. Seguridad

Atiende el atributo de seguridad de la norma ISO/IEC 25010. El sistema debe autenticar a quien lo usa antes de permitir cualquier operación de escritura, y habilitar solo las funciones del rol correspondiente. El diseño contempla tres roles con permisos distintos, descritos en el apartado 11.3, y el caso de uso CU-01 define el flujo de acceso. El criterio de aceptación es que un usuario con rol de consulta no logre registrar un préstamo y que el intento quede rechazado y registrado. En el estado actual del prototipo este requerimiento está diseñado pero no implementado: la autenticación y el control de roles quedaron en el plan de continuidad, porque en esta etapa se priorizó el comportamiento transaccional. Es la brecha más importante que queda abierta.

RNF-02. Integridad de los datos

Atiende la fiabilidad. Los préstamos y las devoluciones se ejecutan como transacciones atómicas con propiedades ACID. En la práctica, la operación de préstamo abarca la selección del ejemplar, el cambio de su estado y la creación del registro del préstamo dentro de una sola transacción; si cualquier paso falla, el motor revierte todo. La selección del ejemplar se hace bajo indicaciones de bloqueo, de modo que dos préstamos simultáneos del mismo título no puedan tomar la misma copia. El criterio de aceptación es que ante un error en un paso intermedio no quede ningún cambio guardado y el ejemplar conserve su estado anterior. Lo comprueban los casos PT-03, PT-04, PT-06 y PT-08.

RNF-03. Veracidad de la información

Atiende la adecuación funcional. Las respuestas del módulo de Inteligencia Artificial se arman solo con los datos que devuelven los nodos, sin estimar disponibilidad. Es una restricción de diseño, no una recomendación: el servicio de interpretación no tiene acceso a las bases de datos y su única salida son criterios de búsqueda sobre el contrato gRPC. El criterio de aceptación es que toda cifra mostrada al usuario coincida con el conteo de ejemplares de la consulta que la originó.

El caso PT-01 verifica, entre otras cosas, que el número de disponibles nunca supere el total de ejemplares de un título.

RNF-04. Eficiencia de desempeño

Atiende la eficiencia de desempeño. La consulta a las tres bibliotecas se ejecuta en paralelo y no una después de otra, de modo que el tiempo total se parezca al de la sede más lenta y no a la suma de las tres. El criterio de aceptación es que, en red local y con hasta cinco mil títulos, la consulta distribuida responda en menos de tres segundos. En la ejecución registrada el caso PT-01 completó la consulta distribuida en 2,09 segundos, incluyendo el tiempo de espera agotado contra los nodos que no estaban levantados.

RNF-05. Disponibilidad

Atiende la fiabilidad, en su subcaracterística de tolerancia a fallos. Si una sede no responde, el sistema sigue operando con las demás e informa cuál falló. Esto es posible porque no hay base central: el cliente consulta a cada nodo por separado y arma el resultado con lo que recibe. El criterio de aceptación es que con un nodo apagado se obtengan los resultados de los otros dos y se señale el que no respondió. Lo comprueba el caso PT-07, que consulta deliberadamente un puerto donde no hay nadie escuchando.

RNF-06. Capacidad de interacción

Atiende la capacidad de interacción. Un bibliotecario sin conocimientos técnicos debe completar las operaciones diarias sin usar la terminal ni escribir consultas SQL. La aplicación de escritorio cubre este requerimiento con un menú de navegación permanente, formularios que validan antes de enviar y mensajes diferenciados para éxito, advertencia y error. El criterio de aceptación es que, tras una inducción de diez minutos, el usuario complete un préstamo y una devolución sin ayuda.

RNF-07. Mantenibilidad

Atiende la mantenibilidad, en su subcaracterística de modularidad. El sistema se organiza en capas, de modo que un cambio de motor de base de datos o de protocolo afecte solo a una de

ellas. El código quedó dividido en presentación, negocio, entidad y datos, con una regla de dependencia en un solo sentido. El criterio de aceptación es que las reglas de negocio no contengan SQL ni referencias al protocolo, lo que se verifica por revisión de código: las sentencias SQL aparecen únicamente en los repositorios de la capa de datos.

RNF-08. Trazabilidad

Atiende la fiabilidad y el soporte a la auditoría. Se conserva el registro de las operaciones que cambian el inventario y de las consultas formuladas al sistema. Las entidades PRESTAMO y DEVOLUCION guardan el historial de cada ejemplar con sus fechas, y las entidades CONSULTA y RESULTADO_CONSULTA guardan lo que el usuario preguntó y lo que respondió cada sede. El criterio de aceptación es que para cualquier ejemplar se pueda reconstruir su secuencia de préstamos y devoluciones con sus fechas. El caso PT-02 recorre esa secuencia completa sobre un título real.

RNF-09. Compatibilidad

Atiende la portabilidad. El sistema funciona sobre Microsoft Windows, Microsoft SQL Server y Python 3.10 o superior, sin hardware especializado. El acceso a la base de datos se hace con un controlador ODBC estándar y las dependencias están declaradas en un archivo de requisitos. El criterio de aceptación es que el prototipo se instale y se ejecute siguiendo únicamente las instrucciones del manual de instalación.

REQUERIMIENTOS DEL SISTEMA

Funcionales y No Funcionales

Nombre del Proyecto:
Sistema Distribuido de Consulta Inteligente de Bibliotecas

Fecha: 05 / 11 / 2026

Grupo: 5

Asignatura: Sistemas Transaccionales

REQUERIMIENTOS FUNCIONALES

Qué debe hacer el sistema

ID	Descripción del requerimiento	Resultado esperado
RF-01	Iniciar sesión. El usuario, el bibliotecario o el administrador ingresa sus credenciales y el sistema le habilita únicamente las funciones de su rol.	Acceso concedido con los permisos del rol correspondiente; los intentos fallidos quedan registrados.
RF-02	Gestionar usuarios. El bibliotecario registra, consulta y actualiza lectores; el administrador los desactiva cuando no tienen préstamos pendientes.	El lector queda registrado con un documento único y sus datos se mantienen al día.
RF-03	Gestionar libros. El bibliotecario registra, busca, modifica y desactiva títulos del catálogo con su ISBN, autor, editorial y año de publicación.	El catálogo de la sede refleja los títulos vigentes, sin ISBN repetidos.
RF-04	Gestionar ejemplares. El bibliotecario asocia copias físicas a un título y cambia su estado entre disponible, mantenimiento y baja.	El inventario muestra cuántas copias hay de cada título y en qué estado está cada una.
RF-05	Consultar disponibilidad. El usuario busca por título, autor o ISBN en una sede o en las tres bibliotecas al mismo tiempo.	Se muestran los ejemplares totales, disponibles y prestados por biblioteca, indicando cuáles sedes respondieron.
RF-06	Registrar préstamo. El bibliotecario entrega un ejemplar disponible a un lector habilitado y el sistema fija la fecha de vencimiento.	El préstamo queda registrado en una sola transacción y el ejemplar pasa a estado prestado.
RF-07	Registrar devolución. El bibliotecario cierra un préstamo activo y anota el estado en que vuelve el ejemplar.	El préstamo queda finalizado y el ejemplar vuelve a estar disponible para futuros préstamos.
RF-08	Controlar préstamos vencidos. El bibliotecario consulta el historial de un lector y el sistema marca los préstamos cuya fecha de vencimiento ya pasó.	Los préstamos vencidos quedan señalados y bloquean nuevos préstamos al mismo lector.
RF-09	Consultar en lenguaje natural. El usuario escribe su pregunta en español y el sistema responde con la disponibilidad real de las tres sedes.	Respuesta redactada solo con datos de los nodos; la pregunta y su resultado quedan en la bitácora de consultas.
RF-10	Generar reportes. El administrador obtiene el inventario por sede y el listado de préstamos vencidos con sus responsables.	Reporte consultable y exportable, con información actualizada del sistema.

REQUERIMIENTOS NO FUNCIONALES

Cómo debe ser el sistema

ID	Descripción del requerimiento	Criterio de aceptación
RNF-01	Seguridad. El sistema autentica a los usuarios antes de cualquier operación de escritura y limita las funciones disponibles según el rol.	Un usuario con rol de consulta no logra registrar un préstamo; el intento queda rechazado y registrado.
RNF-02	Integridad de los datos. Los préstamos y las devoluciones se ejecutan como transacciones atómicas con propiedades ACID.	Ante un error en un paso intermedio no queda ningún cambio guardado y el ejemplar conserva su estado anterior.
RNF-03	Veracidad de la información. Las respuestas del módulo de IA se arman solo con los datos que devuelven los nodos, sin estimar disponibilidad.	Toda cifra mostrada al usuario coincide con el conteo de ejemplares de la consulta que la originó.
RNF-04	Eficiencia de desempeño. La consulta a las tres bibliotecas se ejecuta en paralelo y no una después de otra.	En red local y con hasta 5.000 títulos, la consulta distribuida responde en menos de tres segundos.
RNF-05	Disponibilidad. Si una sede no responde, el sistema sigue operando con las demás e informa cuál falló.	Con un nodo apagado se obtienen los resultados de los otros dos y se señala el que no respondió.
RNF-06	Capacidad de interacción. Un bibliotecario sin conocimientos técnicos completa las operaciones diarias sin usar la terminal ni escribir consultas SQL.	Tras una inducción de diez minutos, el usuario completa un préstamo y una devolución sin ayuda.
RNF-07	Mantenibilidad. El sistema se organiza en capas, de modo que un cambio de motor de base de datos o de protocolo afecte solo a una de ellas.	Las reglas de negocio no contienen SQL ni referencias al protocolo; se verifica por revisión de código.
RNF-08	Trazabilidad. Se conserva el registro de las operaciones que cambian el inventario y de las consultas formuladas al sistema.	Para cualquier ejemplar se reconstruye su secuencia de préstamos y devoluciones con sus fechas.
RNF-09	Compatibilidad. Funciona sobre Microsoft Windows, Microsoft SQL Server y Python 3.10 o superior, sin hardware especializado.	El prototipo se instala y se ejecuta siguiendo únicamente las instrucciones del manual de instalación.

12.4. Trazabilidad entre requerimientos, casos de uso y modelo de datos

Un requerimiento que no se puede seguir hasta el diseño y hasta una prueba es difícil de defender. La Tabla 4 enlaza cada requerimiento funcional con el caso de uso que lo desarrolla, las entidades del modelo relacional que toca, la operación del contrato gRPC que lo implementa y la evidencia que lo comprueba. Las operaciones distribuidas, que son las que involucran a los tres nodos a la vez, aparecen señaladas en la última columna.

Tabla 4

Matriz de trazabilidad de los requerimientos funcionales

RF	Caso de uso	Entidades del modelo relacional	Operación del contrato gRPC	Evidencia	Alcance
RF-01	CU-01	USUARIO	Pendiente de implementar	—	Local al nodo
RF-02	CU-01, CU-05	USUARIO, PRESTAMO	ListarUsuarios	PT-03	Local al nodo
RF-03	CU-05	LIBRO	ConsultarDisponibilidad	PT-01	Local al nodo
RF-04	CU-05	EJEMPLAR, LIBRO	ListarEjemplares	PT-01, PT-03	Local al nodo
RF-05	CU-02, CU-03	LIBRO, EJEMPLAR, BIBLIOTECA	ConsultarDisponibilidad	PT-01, PT-02, PT-07	**Distribuida**
RF-06	CU-06	PRESTAMO, EJEMPLAR, USUARIO	RegistrarPrestamo	PT-03, PT-06, PT-08	Local, con efecto visible en la vista global
RF-07	CU-07	DEVOLUCION, PRESTAMO, EJEMPLAR	RegistrarDevolucion	PT-04, PT-05	Local, con efecto visible en la vista global
RF-08	CU-06, CU-07	PRESTAMO, USUARIO	ListarPrestamos	PT-03, PT-04	Local al nodo
RF-09	CU-04	CONSULTA, RESULTADO_CONSULTA, LIBRO, EJEMPLAR	ConsultarDisponibilidad, con criterios extraídos por el modelo	PT-01	**Distribuida**
RF-10	CU-08	EJEMPLAR, PRESTAMO, BIBLIOTECA	ConsultarDisponibilidad, ListarPrestamos	PT-01	**Distribuida**

Nota. Elaboración propia. Los casos PT corresponden a la suite de pruebas transaccionales del apartado 19. Las operaciones de escritura son siempre locales al nodo que es dueño del ejemplar: no existe ninguna transacción que abarque dos bases de datos, decisión que se explica en el apartado 13.2.

13. Arquitectura de software

13.1. Visión general

La estructura conceptualmente quedaría como algo así:

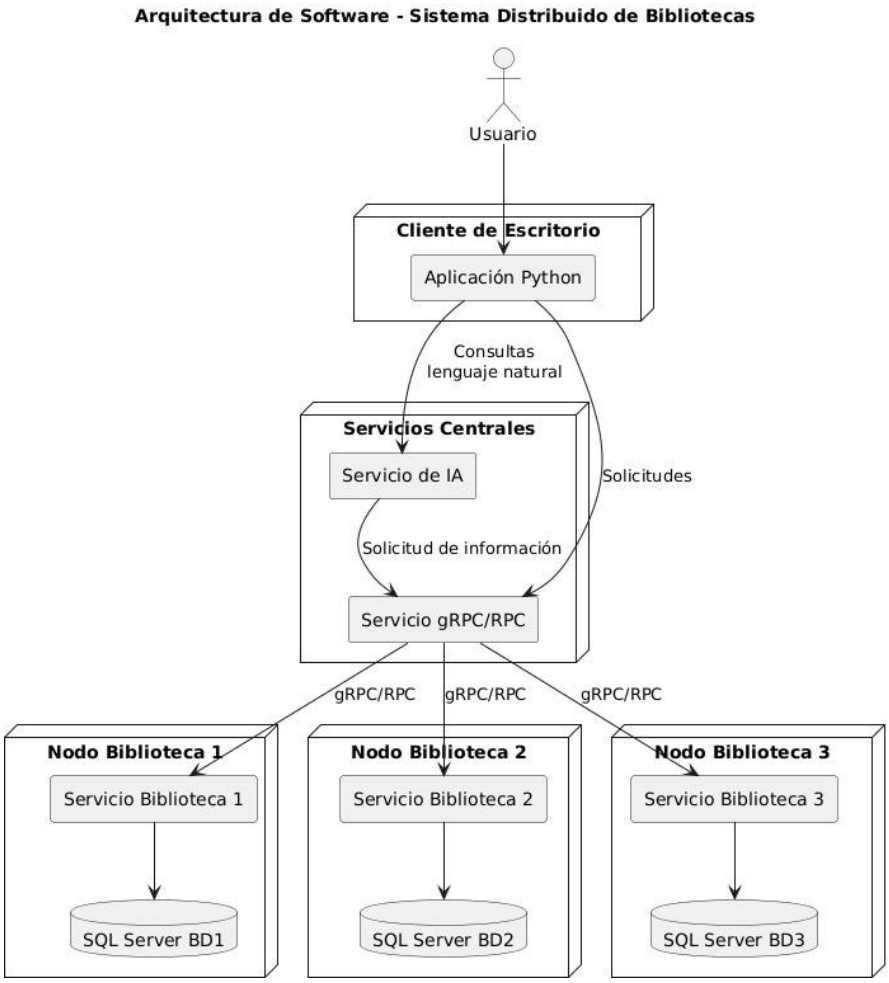


Figura 1. Arquitectura de software del sistema distribuido de bibliotecas.

El usuario interactúa con el cliente de escritorio desarrollado en Python. Las consultas en lenguaje natural se dirigen al servicio de Inteligencia Artificial, que las traduce en solicitudes de información; el servicio gRPC/RPC las distribuye hacia los tres nodos. Cada nodo ejecuta su propio servicio de biblioteca sobre una instancia independiente de SQL Server, de modo que ninguna sede depende de las demás para operar.

Dentro de cada nodo el código se organiza en cuatro capas con dependencia en un solo sentido. La capa de presentación expone el contrato gRPC y la aplicación de escritorio; la capa de negocio contiene las reglas de préstamo y devolución; la capa de entidad define los objetos del dominio; y la capa de datos agrupa los repositorios y la conexión a SQL Server. Las sentencias SQL viven únicamente en la capa de datos, que es la condición que exige el requerimiento RNF-07.

13.2. Operación distribuida: tiempos de espera y errores parciales

Un sistema repartido en tres nodos tiene que decidir por anticipado qué hace cuando alguno no contesta. Esta sección recoge esas decisiones, porque son las que explican por qué el sistema sigue siendo confiable aunque una sede esté caída.

Dónde ocurre cada operación

La primera decisión es la más importante y es también la que evita la mayoría de los problemas: **no existe ninguna transacción que abarque dos bases de datos**. Cada ejemplar pertenece a una sola biblioteca, y el préstamo o la devolución de ese ejemplar se ejecuta por completo dentro del nodo que lo tiene. No se usa transacción distribuida ni confirmación en dos fases. Lo único que es realmente distribuido es la lectura: la consulta de disponibilidad, que se arma preguntando a las tres sedes y sumando lo que responden.

La consecuencia es que una caída solo puede dejar una consulta incompleta, nunca un inventario inconsistente. Si el nodo dueño del ejemplar está disponible, la operación de escritura se confirma con todas las garantías ACID del motor; si no lo está, la operación sencillamente no se intenta y el usuario recibe el aviso correspondiente.

Tiempos de espera

Cada llamada al contrato gRPC lleva un tiempo límite de cinco segundos. Ese valor cubre con holgura una consulta en red local y, al mismo tiempo, evita que la aplicación quede bloqueada esperando a un equipo apagado. Cuando el límite se agota, la biblioteca de gRPC levanta un

error que el cliente atrapa y convierte en una respuesta marcada como fallida, con el motivo incluido.

Las tres sedes se consultan en paralelo, cada una en su propio hilo, y los resultados se recogen a medida que llegan. Por eso el tiempo total de una consulta distribuida se parece al de la sede más lenta y no a la suma de las tres, que es lo que exige el requerimiento RNF-04. En la ejecución registrada en el apartado 19, una consulta con un solo nodo levantado y dos agotando su tiempo límite tardó 2,09 segundos en total.

Errores parciales

Cuando un nodo falla, su respuesta se guarda con la marca de fallida y el motivo que devolvió gRPC, y el resultado global se arma con las respuestas que sí llegaron. El sistema nunca descarta la consulta entera porque una sede no contestó. La aplicación muestra entonces dos cosas al mismo tiempo: los datos de las sedes que respondieron y un aviso que nombra las que no, con el texto «Nodos sin respuesta» seguido de sus números.

Esa distinción es deliberada y responde al requerimiento RNF-03. Una consulta que no alcanzó a las tres sedes es una respuesta incompleta, no una respuesta negativa: que el sistema no encuentre un título en las dos bibliotecas que respondieron no significa que no exista en la tercera. Presentar las dos situaciones como iguales sería dar por cierta una información que el sistema no tiene.

Detección de caídas y recuperación

La aplicación mantiene un monitor que consulta el estado de los tres nodos cada seis segundos, en segundo plano, mediante la operación de verificación del contrato. Cuando el estado de alguno cambia, la barra inferior se actualiza sola y aparece un aviso que indica qué nodo dejó de responder o cuál volvió. El usuario no tiene que cambiar de pantalla ni reiniciar la aplicación para enterarse.

La recuperación no necesita ninguna acción especial. Como no hay datos replicados, un nodo que vuelve a estar en línea no tiene nada que poner al día: a partir de la siguiente consulta sus

datos vuelven a aparecer en el resultado agregado. Esta es la ventaja práctica de haber descartado la replicación.

Códigos de error y mensajes al usuario

Las operaciones rechazadas no se señalan con mensajes de texto libre sino con los códigos de estado del propio protocolo, que el cliente traduce a un mensaje entendible. Un préstamo sobre un usuario o un libro que no existe devuelve NOT_FOUND; un intento de devolver dos veces el mismo préstamo devuelve FAILED_PRECONDITION; un título sin ejemplares libres devuelve RESOURCE_EXHAUSTED; y un nodo inalcanzable produce un error de transporte. El caso PT-05 comprueba los tres primeros y verifica además que el servidor siga en pie después de los rechazos.

Las operaciones de escritura no se reintentan de forma automática cuando el tiempo de espera se agota. La razón es que el cliente no puede distinguir entre una petición que nunca llegó y una que se ejecutó pero cuya respuesta se perdió, y un reintento en el segundo caso registraría dos préstamos del mismo ejemplar. Ante esa duda el sistema informa al usuario y deja la decisión en sus manos, que es la opción segura mientras el contrato no incluya identificadores de idempotencia.

14. Diagrama de casos de uso UML



Figura 2. Diagrama de casos de uso del sistema distribuido de consulta inteligente de libros.

CU-01. Iniciar sesión

Campo	Descripción
Nombre	Iniciar sesión
Actor principal	Usuario, Bibliotecario o Administrador
Objetivo	Permitir que los usuarios accedan al sistema de acuerdo con sus credenciales y tipo de usuario.
Precondiciones	El usuario debe estar registrado y activo en el sistema.
Flujo principal	1. El usuario ingresa sus credenciales. 2. El sistema valida la información. 3. El sistema identifica el tipo de usuario. 4. El sistema permite el acceso a las funcionalidades correspondientes.
Resultado esperado	El usuario accede correctamente al sistema y puede utilizar las funciones permitidas según su tipo.

CU-02. Consultar catálogo distribuido

Campo	Descripción
Nombre	Consultar catálogo distribuido
Actor principal	Usuario
Objetivo	Consultar un libro en las tres bibliotecas desde una única aplicación.
Precondiciones	El usuario debe haber iniciado sesión y los nodos de las bibliotecas deben estar disponibles para realizar la consulta.
Flujo principal	1. El usuario ingresa el título, autor o ISBN del libro. 2. El sistema recibe la solicitud. 3. La solicitud se envía mediante el servicio gRPC/RPC. 4. Los tres nodos consultan sus respectivas bases de datos. 5. Cada biblioteca devuelve los resultados encontrados. 6. El sistema reúne y presenta los resultados.
Flujo alternativo	4a. Si una biblioteca no responde dentro del tiempo límite, el sistema presenta los resultados de las sedes restantes e indica cuál no respondió, advirtiendo que la consulta está incompleta.
Resultado esperado	El usuario visualiza en una sola consulta las bibliotecas donde existe el libro y su disponibilidad.

CU-03. Consultar disponibilidad

Campo	Descripción
Nombre	Consultar disponibilidad
Actor principal	Usuario
Objetivo	Conocer la cantidad y ubicación de ejemplares disponibles de un libro.
Precondiciones	El libro debe existir en al menos una de las bibliotecas o haber sido consultado previamente.
Flujo principal	1. El usuario selecciona un libro. 2. El sistema consulta los nodos de las bibliotecas. 3. Cada nodo verifica el estado de sus ejemplares. 4. El sistema recopila la información. 5. Se muestra la disponibilidad por biblioteca.
Resultado esperado	El usuario conoce en qué biblioteca está disponible el libro y la cantidad de ejemplares disponibles.

CU-04. Realizar consulta en lenguaje natural

Campo	Descripción
Nombre	Realizar consulta en lenguaje natural
Actor principal	Usuario
Objetivo	Permitir que el usuario pregunte por un libro en lenguaje corriente y obtenga una respuesta redactada con la disponibilidad real de las tres bibliotecas.
Precondiciones	El usuario debe haber iniciado sesión y el servicio de Inteligencia Artificial debe estar disponible.
Flujo principal	1. El usuario escribe su pregunta en lenguaje natural. 2. El servicio de IA interpreta la pregunta e identifica el título, el autor o el ISBN buscado. 3. El sistema ejecuta la consulta de disponibilidad sobre los tres nodos. 4. Cada biblioteca devuelve el conteo de ejemplares disponibles. 5. El servicio de IA redacta la respuesta utilizando únicamente esos datos. 6. La pregunta y los resultados se almacenan en la bitácora de consultas.
Flujo alternativo	2a. Si la pregunta resulta ambigua o no se identifica ningún criterio de búsqueda, el sistema informa al usuario y le solicita que precise su consulta, sin presentar resultados aproximados.
Resultado esperado	El usuario obtiene una respuesta en lenguaje natural que indica en qué bibliotecas hay ejemplares disponibles del libro consultado.

CU-05. Gestionar libros

Campo	Descripción
Nombre	Gestionar libros
Actor principal	Bibliotecario
Objetivo	Registrar, modificar, consultar y desactivar información de los libros y sus ejemplares.
Precondiciones	El bibliotecario debe haber iniciado sesión y contar con permisos para gestionar el catálogo.
Flujo principal	1. El bibliotecario selecciona la gestión de libros. 2. El sistema muestra las opciones disponibles. 3. El bibliotecario registra, modifica, consulta o desactiva un libro. 4. El sistema valida la información. 5. El sistema almacena los cambios en la base de datos correspondiente.
Resultado esperado	La información del libro queda registrada o actualizada correctamente en la biblioteca correspondiente.

CU-06. Registrar préstamo

Campo	Descripción
Nombre	Registrar préstamo
Actor principal	Bibliotecario
Objetivo	Registrar el préstamo de un ejemplar a un usuario y actualizar su disponibilidad.
Precondiciones	El usuario debe estar registrado y activo. El ejemplar debe existir y encontrarse disponible.
Flujo principal	1. El bibliotecario identifica al usuario. 2. Selecciona el ejemplar que será prestado. 3. El sistema verifica que el usuario esté habilitado. 4. El sistema verifica la disponibilidad del ejemplar. 5. Se registra el préstamo. 6. Se actualiza el estado del ejemplar como no disponible. 7. La operación se confirma mediante una transacción.
Flujo alternativo	4a. Si otro bibliotecario tomó el último ejemplar mientras se diligenciaba la operación, el sistema rechaza el préstamo con el código RESOURCE_EXHAUSTED y no deja ningún registro parcial.
Resultado esperado	El préstamo queda registrado y el ejemplar pasa a estado no disponible.

CU-07. Registrar devolución

Campo	Descripción
Nombre	Registrar devolución
Actor principal	Bibliotecario
Objetivo	Registrar la devolución de un ejemplar y actualizar nuevamente su disponibilidad.
Precondiciones	Debe existir un préstamo activo asociado al ejemplar.
Flujo principal	1. El bibliotecario identifica el préstamo. 2. El sistema consulta la información del préstamo. 3. Se registra la fecha de devolución. 4. El préstamo cambia a estado finalizado. 5. El ejemplar cambia a estado disponible. 6. La operación se confirma mediante una transacción.
Resultado esperado	La devolución queda registrada y el ejemplar vuelve a estar disponible para futuros préstamos.

CU-08. Generar reportes

Campo	Descripción
Nombre	Generar reportes
Actor principal	Administrador
Objetivo	Obtener información consolidada sobre libros, disponibilidad, préstamos y devoluciones.
Precondiciones	El administrador debe haber iniciado sesión y existir información registrada en el sistema.
Flujo principal	1. El administrador selecciona el tipo de reporte. 2. El sistema consulta la información necesaria. 3. Se recopilan los datos de las bibliotecas correspondientes. 4. El sistema procesa la información. 5. Se genera y presenta el reporte.
Resultado esperado	El administrador obtiene un reporte con información actualizada del sistema.

15. Diagrama de clases UML

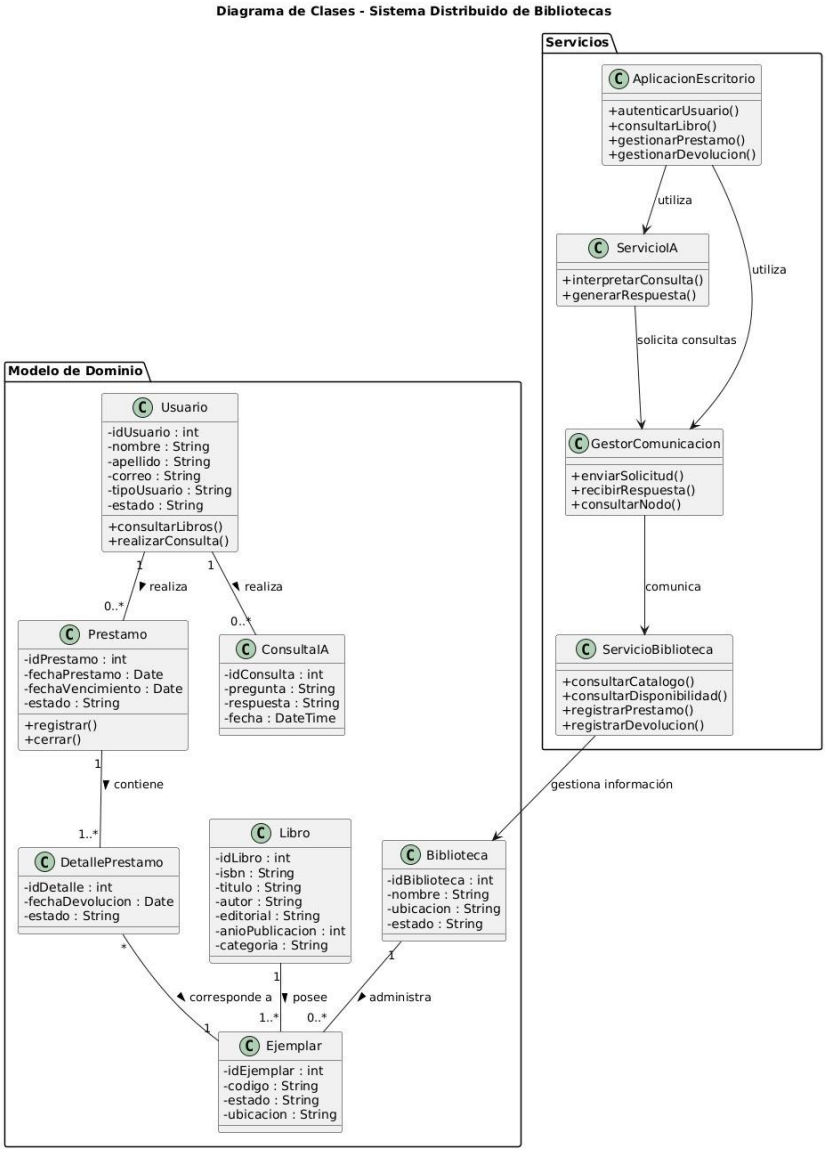


Figura 3. Diagrama de clases del sistema distribuido de bibliotecas.

16. Modelo relacional

Las relaciones entre las entidades del modelo son las siguientes:

- BIBLIOTECA 1 ——> N EJEMPLAR
- LIBRO 1 ——> N EJEMPLAR
- USUARIO 1 ——> N PRESTAMO
- EJEMPLAR 1 ——> N PRESTAMO
- PRESTAMO 1 ——> 0..1 DEVOLUCION
- CONSULTA 1 ——> N RESULTADO_CONSULTA
- BIBLIOTECA 1 ——> N RESULTADO_CONSULTA

Conviene subrayar una consecuencia de este modelo que explica buena parte del diseño distribuido: el esquema se instancia completo en cada una de las tres bases de datos. La entidad USUARIO, en particular, es local a cada sede, de modo que un mismo código de usuario puede corresponder a personas distintas en bibliotecas distintas y un lector solo puede recibir préstamos en la sede donde está inscrito. No existe ninguna llave foránea que cruce de una base de datos a otra, y esa es precisamente la razón por la que ninguna transacción necesita abarcar dos nodos.

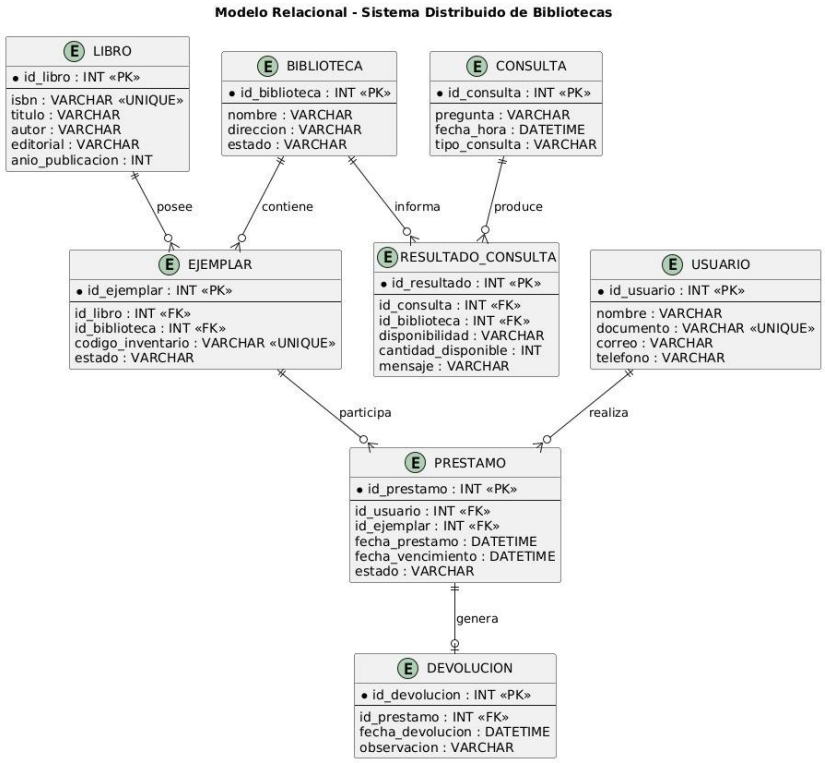


Figura 4. Modelo relacional del sistema distribuido de bibliotecas.

17. Marco integrador: servicio de Inteligencia Artificial

17.1. Servicio de Inteligencia Artificial propuesto

Se propone usar un servicio de consulta en lenguaje natural apoyado en un modelo de lenguaje de gran tamaño, al que se accede por la interfaz de programación de un proveedor externo. El servicio se usa solo para traducir lo que el usuario quiere. Recibe la pregunta escrita en español y devuelve de forma estructurada qué está buscando, sea el título, el autor o el ISBN. Con eso el sistema arma la consulta hacia los nodos.

Se dejó prevista una segunda alternativa, con reglas lingüísticas locales hechas con expresiones regulares y un diccionario de términos de biblioteca, que no necesita internet ni servicios de pago. Las dos alternativas cumplen el mismo contrato dentro del sistema, así que escoger una u otra no obliga a cambiar nada más. La razón de tener las dos es práctica: el día de la sustentación no se puede dar por seguro que haya conexión ni credenciales disponibles.

El apartado 18 desarrolla una tercera opción, que es la que atiende el componente de red neuronal del proyecto: un modelo propio, entrenado sobre el dominio bibliográfico, que se ejecuta en el equipo y cumple el mismo contrato que las dos anteriores.

17.2. Función dentro del proyecto

El servicio cubre la distancia que hay entre cómo pregunta el usuario y cómo puede buscar el sistema. El usuario dice «¿tienen Cien años de soledad?» o «¿dónde puedo encontrar algo de García Márquez?», mientras que el sistema necesita un criterio concreto, sea título, autor o ISBN, para consultar las tres bibliotecas. Esa traducción es el trabajo del servicio de Inteligencia Artificial.

Un formulario de búsqueda deja por fuera varias situaciones que en la práctica son comunes. El usuario muchas veces no sabe si lo que recuerda es el título o el autor. Suele acordarse del nombre a medias, o parecido. Y para él consultar el catálogo y consultar la disponibilidad son la misma pregunta, aunque para el sistema sean dos cosas distintas. Affolter et al. (2019) ya señalaban que la ambigüedad de lo que escribe el usuario es la dificultad principal de estas

interfaces, y que acotar el dominio, en este caso a los campos de un catálogo bibliográfico, es la forma usual de controlarla.

17.3. Integración con el sistema

El servicio queda entre la aplicación de escritorio y el servicio de comunicación, como se ve en la Figura 1. No entra a las bases de datos, no genera SQL y no sabe cómo están armados los nodos por dentro. Lo único que produce son los criterios de búsqueda que el contrato gRPC ya acepta. El flujo tiene cinco pasos.

Tabla 5

Flujo de integración del servicio de Inteligencia Artificial

Paso	Acción	Descripción
1	Recepción de la pregunta	El usuario escribe su consulta en lenguaje natural en la aplicación de escritorio.
2	Interpretación	El servicio de IA analiza el enunciado y devuelve una intención estructurada con los campos de búsqueda que logró identificar.
3	Consulta distribuida	El sistema ejecuta la operación de disponibilidad sobre las tres bibliotecas en paralelo, con los criterios obtenidos.
4	Redacción de la respuesta	El servicio compone la respuesta en lenguaje natural utilizando únicamente las cifras devueltas por los nodos.
5	Registro	La pregunta y el resultado de cada biblioteca se almacenan en la bitácora de consultas, en las entidades CONSULTA y RESULTADO_CONSULTA del modelo relacional.

Nota. Elaboración propia. El flujo corresponde al caso de uso CU-04 del apartado 14.

Restricción de diseño. El servicio de Inteligencia Artificial no decide ni calcula la disponibilidad de ningún título. Solo interpreta la pregunta y redacta la respuesta; las cifras siempre salen de las bases de datos de los nodos. Si la interpretación falla o la pregunta queda ambigua, el sistema tiene que decírselo al usuario y pedirle que precise, en vez de mostrarle un resultado aproximado. Esta regla quedó escrita en el requerimiento RNF-03 y en el flujo alternativo del caso de uso CU-04.

17.4. Resultado esperado

Para el usuario, la ganancia es resolver en una sola consulta lo que hoy le exige ir o llamar a tres sedes, preguntando con sus propias palabras y recibiendo una respuesta que le dice dónde hay un ejemplar libre.

Para el personal de las bibliotecas, el servicio quita buena parte de las consultas sencillas que hoy se atienden en el mostrador o por teléfono, y deja tiempo para lo que sí requiere criterio profesional.

Para la organización, la bitácora de consultas deja ver qué están buscando los usuarios, incluidas las búsquedas que no encontraron nada. Esa información hoy se pierde, y sirve directamente para decidir qué títulos comprar y cómo repartir los ejemplares entre sedes.

17.5. Límites de esta propuesta

En el estado actual del prototipo el componente que interpreta la pregunta no está implementado. Lo que sí está construido y probado es todo lo que lo rodea: la interfaz del servicio dentro del código, con sus estructuras de intención y de respuesta; la consulta de disponibilidad distribuida sobre la que se apoya, verificada en el apartado 19; y la pantalla de consulta, que hoy funciona por coincidencia de texto y le advierte al usuario que la respuesta no proviene todavía del módulo de Inteligencia Artificial. Lo que falta es el intérprete, y el apartado siguiente describe cómo se va a construir.

18. Red neuronal para la interpretación de consultas

18.1. Qué problema resuelve la red y por qué hace falta

La pantalla de consulta funciona hoy por coincidencia de texto: toma la pregunta, le quita las palabras vacías y busca lo que queda contra los títulos y autores del catálogo. Funciona cuando el usuario escribe el título casi exacto y falla en todo lo demás. No distingue si «García Márquez» es un autor o parte de un título, no entiende que «¿lo tienen en la sede del norte?» se refiere a una biblioteca concreta, y no sabe decir que una pregunta quedó fuera de su alcance: siempre devuelve el resultado más parecido, aunque no tenga nada que ver.

Ese último punto es el que obliga a cambiar de técnica. El requerimiento RNF-03 exige que el sistema no muestre resultados aproximados cuando no entendió la pregunta, sino que pida precisión. Para cumplirlo el componente que interpreta tiene que poder medir qué tan seguro está, y una búsqueda por coincidencia de texto no produce esa medida. Una red neuronal sí: entrega una distribución de probabilidad sobre las interpretaciones posibles, y con ella se puede fijar un umbral por debajo del cual el sistema prefiere preguntar antes que responder.

18.2. Formulación de la tarea

Interpretar la pregunta se descompone en dos tareas que se resuelven a la vez sobre el mismo enunciado:

- **Clasificación de la intención.** Decidir qué quiere el usuario, entre un conjunto cerrado de intenciones. Es una clasificación de una sola etiqueta sobre la frase completa.
- **Etiquetado de entidades.** Marcar qué parte del enunciado es el título, cuál el autor, cuál el ISBN y cuál la biblioteca. Es una clasificación token por token, con el esquema BIO, donde cada palabra recibe la etiqueta de comienzo de una entidad, de continuación, o de ninguna.

Chen, Zhuo y Wang (2019) mostraron que conviene resolver las dos tareas con un solo modelo y no con dos separados, porque se condicionan mutuamente: saber que la intención es «buscar por

autor» ayuda a etiquetar el nombre propio que aparece en la frase, y al revés. Esa es la formulación que adopta este diseño.

La Tabla 6 recoge las intenciones y las entidades definidas para el dominio bibliográfico del proyecto.

Tabla 6

Intenciones y entidades definidas para el dominio del proyecto

Etiqueta	Significado	Ejemplo de enunciado
Intenciones		
consultar_disponibilidad	Pregunta si hay ejemplares libres de un título.	«¿Tienen Cien años de soledad?»
buscar_por_autor	Pide obras de un autor, sin título concreto.	«¿Qué tienen de Borges?»
buscar_por_isbn	Identifica el libro por su código normalizado.	«Busco el 9780132350884.»
ubicar_sede	Pregunta en cuál de las tres bibliotecas está.	«¿En qué sede hay Clean Code?»
consultar_prestamos	Pide el estado de los préstamos de un lector.	«¿Qué libros tengo prestados?»
fuera_de_dominio	La pregunta no corresponde al catálogo.	«¿A qué hora cierran?»
Entidades		
TITULO	Nombre de la obra.	«¿Tienen Cien años de soledad?»
AUTOR	Nombre de la persona que escribió la obra.	«algo de Gabriel García Márquez»
ISBN	Código normalizado de trece o diez dígitos.	«el 9780307474728»
BIBLIOTECA	Sede a la que se refiere la pregunta.	«en la Biblioteca 2»

Nota. Elaboración propia. La intención *fuera_de_dominio* es la que permite que el sistema reconozca que una pregunta no le corresponde, en lugar de forzar una búsqueda.

18.3. Arquitectura propuesta

El modelo sigue la arquitectura conjunta de Chen et al. (2019), adaptada al español. Consta de un codificador preentrenado y dos capas de salida que comparten ese codificador, como se muestra en la Figura 5.

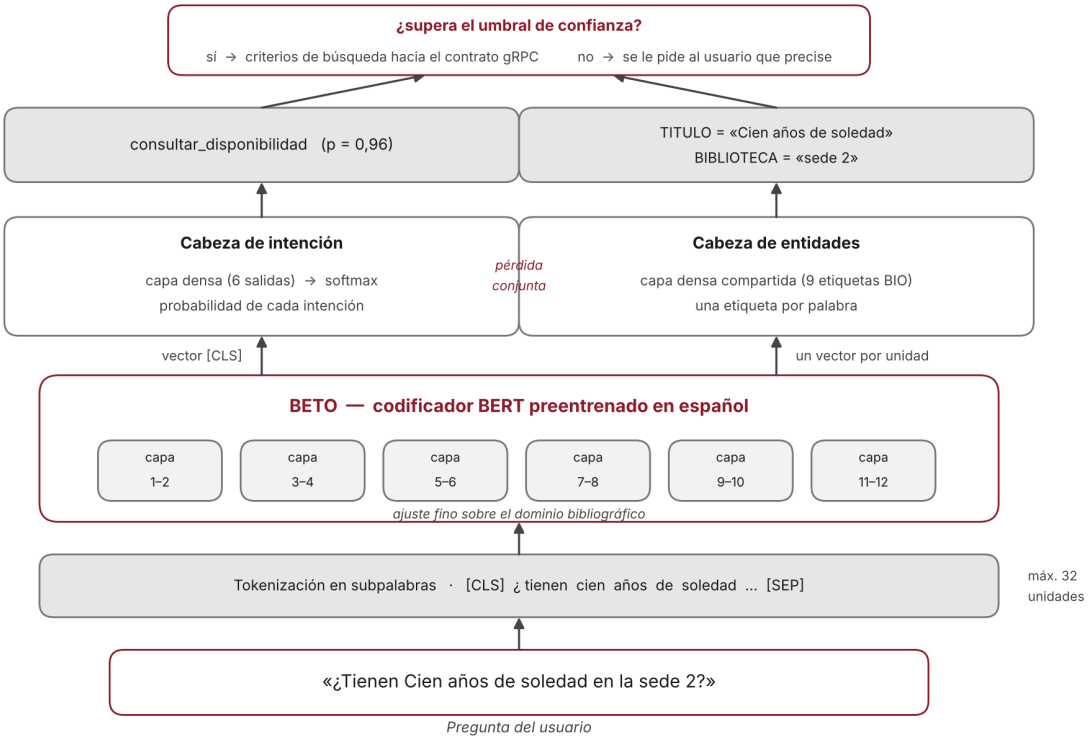


Figura 5. Arquitectura de la red neuronal propuesta para la interpretación de consultas.

Capa de entrada. La pregunta se divide en unidades de subpalabra y se le anteponen los marcadores habituales de BERT. Las entradas se truncan a treinta y dos unidades, longitud que cubre con holgura las preguntas que se observan en el mostrador de una biblioteca.

Codificador. Se usa BETO, un modelo BERT preentrenado exclusivamente con textos en español por el Departamento de Ciencias de la Computación de la Universidad de Chile (Cañete et al., 2023). Preentrenar en español importa: los modelos multilingües reparten su vocabulario entre más de cien idiomas, y Cañete et al. encontraron que un modelo dedicado al español supera a los multilingües en la mayoría de las tareas evaluadas. El codificador produce un vector por cada unidad de entrada más un vector que resume la frase completa, siguiendo el esquema original de BERT (Devlin et al., 2019).

Cabeza de intención. Toma el vector que resume la frase, lo pasa por una capa densa de seis salidas, una por intención, y aplica una función softmax. La salida es la probabilidad de cada intención, y de ahí sale la medida de confianza que necesita el sistema.

Cabeza de entidades. Toma el vector de cada unidad y lo pasa por una capa densa compartida que produce las nueve etiquetas BIO, dos por cada tipo de entidad más la etiqueta de «ninguna». Solo se etiqueta la primera unidad de cada palabra, como hace el modelo de referencia.

Función de pérdida. Las dos tareas se entrenan juntas sumando sus pérdidas de entropía cruzada, de modo que el ajuste del codificador sirva a ambas.

18.4. Conjunto de datos

No existe un conjunto de datos público de preguntas de biblioteca en español, así que hay que construirlo. El plan combina dos fuentes, que es el procedimiento habitual cuando el dominio es cerrado y los datos reales escasean:

- **Generación a partir del catálogo real.** Se definen plantillas de pregunta («¿tienen {titulo}?», «¿qué hay de {autor}?», «¿en qué sede está {titulo}?») y se rellenan con los títulos y autores que ya están cargados en las tres bases de datos. Esto produce varios miles de enunciados con sus etiquetas puestas automáticamente, sin trabajo manual y sin riesgo de error de anotación.
- **Preguntas escritas a mano.** Las plantillas producen frases demasiado correctas. Para que el modelo aguante lo que la gente escribe de verdad se añade un bloque anotado manualmente con preguntas incompletas, con errores de tildes, con el título mal recordado y con preguntas fuera de dominio. Este bloque es el que más aporta a la robustez y el que se reserva íntegro para la evaluación.

La partición prevista es de setenta por ciento para entrenamiento, quince para validación y quince para prueba, cuidando que ningún título aparezca a la vez en entrenamiento y en prueba. Esa separación es importante: si el mismo título está en los dos lados, el modelo puede memorizarlo y las métricas quedan infladas, que es justamente el problema que Yu et al. (2018) buscaban evitar al construir Spider con bases de datos distintas en cada partición.

18.5. Entrenamiento y evaluación

El entrenamiento consiste en un ajuste fino del modelo preentrenado, no en entrenar desde cero. Esa es la razón por la que el proyecto es viable sin infraestructura especializada: el conocimiento del idioma ya está en BETO y solo hay que enseñarle la tarea del dominio, lo que se consigue con unas pocas pasadas sobre el conjunto de entrenamiento.

Las métricas de evaluación son las tres que usa el trabajo de referencia, porque permiten comparar contra resultados publicados:

- **Exactitud de intención.** Porcentaje de preguntas cuya intención se clasificó bien.
- **Medida F1 de entidades.** Media armónica entre precisión y exhaustividad sobre las entidades extraídas.
- **Exactitud a nivel de enunciado.** Porcentaje de preguntas en las que *todo* salió bien a la vez: la intención y todas las entidades. Es la métrica exigente y la que de verdad importa aquí, porque una sola entidad mal extraída ya produce una consulta equivocada.

A estas tres se añade una métrica operativa propia del proyecto: el porcentaje de preguntas resueltas sin tener que pedirle al usuario que precise. Mide la utilidad real del componente, porque un modelo que siempre pide aclaración nunca se equivoca pero tampoco sirve.

La Tabla 7 recoge los resultados que la arquitectura de referencia obtuvo en los dos conjuntos de datos estándar del área, frente al mejor modelo anterior. Sirven como expectativa razonable de lo que puede alcanzar el modelo del proyecto, teniendo en cuenta que su dominio es bastante más estrecho.

Tabla 7

Resultados de referencia de la arquitectura conjunta sobre los conjuntos ATIS y Snips

Modelo	**Snips**			**ATIS**		
	Intención	**F1 entidades**	**Enunciado**	**Intención**	**F1 entidades**	**Enunciado**
BERT conjunto	98,6	97,0	92,8	97,5	96,1	88,2
BERT conjunto + CRF	98,4	96,7	92,6	97,9	96,0	88,6
Slot-Gated (modelo anterior)	97,0	88,8	75,5	94,1	95,2	82,6

Nota. Valores en porcentaje, tomados de Chen et al. (2019). El conjunto ATIS contiene 4.478 enunciados de entrenamiento con 21 intenciones y 120 etiquetas de entidad; Snips contiene 13.084 enunciados con 7 intenciones y 72 etiquetas. La mejora más notoria está en la exactitud a nivel de enunciado sobre Snips, que pasa de 75,5 a 92,8.

18.6. Integración con el resto del sistema

La red no cambia nada del sistema distribuido, y eso es intencional. Su salida es la misma estructura de intención que ya consumen las otras dos alternativas descritas en el apartado 17, de modo que los nodos, el contrato gRPC y las bases de datos quedan exactamente igual. El modelo se ejecuta en el equipo donde corre la aplicación, sin salir a internet.

El umbral de confianza es la pieza que conecta la red con el requerimiento RNF-03. Si la probabilidad de la intención más votada no supera el valor fijado, el sistema no consulta a los nodos: le dice al usuario que no entendió bien y le pide que precise, siguiendo el flujo alternativo del caso de uso CU-04. El valor concreto del umbral se fija sobre el conjunto de validación, buscando el punto donde el error de interpretación baje a un nivel aceptable sin que el sistema empiece a pedir aclaración todo el tiempo.

Sobre el tiempo de respuesta, el requerimiento RNF-04 fija un límite de tres segundos para la consulta completa, de los cuales la consulta distribuida ya consume alrededor de dos. La interpretación tiene que caber en lo que queda. Un modelo del tamaño de BETO sobre una entrada de treinta y dos unidades responde en un procesador corriente en un tiempo del orden de las décimas de segundo, de modo que el presupuesto alcanza; de todos modos, la medición del tiempo de inferencia queda incluida como criterio de aceptación del componente.

18.7. Límites y riesgos

Conviene dejar escritos los riesgos de esta propuesta, porque son conocidos y tienen respuesta prevista.

El primero es que los datos generados con plantillas no se parecen lo suficiente a las preguntas reales y el modelo funcione bien en la prueba y mal en el mostrador. La respuesta es el bloque anotado a mano, que se reserva completo para evaluación, y la bitácora de consultas del sistema, que irá acumulando preguntas reales para reentrenar más adelante.

El segundo es que el catálogo cambia. Un título nuevo no está en los datos de entrenamiento, pero la red no necesita haberlo visto: aprende a reconocer *dónde* está el título dentro de la frase, no cuáles títulos existen. La verificación de si ese título existe la hacen los nodos, como siempre.

El tercero es que la red se equivoque con confianza alta. Es el riesgo más serio de cualquier modelo de este tipo y no se elimina del todo. Lo que lo contiene es la restricción de diseño del apartado 17: aunque la interpretación sea errónea, las cifras que ve el usuario siguen saliendo de las bases de datos, así que el peor resultado posible es una búsqueda que no encuentra lo que el usuario quería, nunca un dato de disponibilidad inventado.

Por último, este componente no está implementado en la presente entrega. Lo que queda definido aquí es el diseño completo, el conjunto de etiquetas, el procedimiento para construir los datos y las métricas con las que se va a juzgar. La alternativa de reglas lingüísticas del apartado 17 sigue disponible como respaldo, con el mismo contrato, de modo que el sistema funciona aunque la red no llegue a entrenarse.

19. Evidencias de pruebas transaccionales

19.1. Alcance y método

Las garantías que promete el apartado 12 no se sostienen con una afirmación: hay que mostrarlas funcionando. Para eso se construyó una suite de pruebas que se ejecuta contra los nodos reales, con SQL Server por debajo, y que deja un informe fechado con el detalle de cada comprobación.

La suite no comprueba funciones por separado sino operaciones completas de extremo a extremo: sale de la aplicación cliente, atraviesa el contrato gRPC, llega a las reglas de negocio, toca la base de datos y vuelve. Esa es la razón de que sirva como evidencia del comportamiento transaccional y no solo del código.

Cada caso declara de antemano qué espera observar y lo compara con lo que obtuvo. Un caso pasa solo si todas sus comprobaciones coinciden. La suite deja además rastro de los valores concretos que encontró, de modo que el informe se pueda revisar sin volver a ejecutarla.

Las pruebas son repetibles y no dejan residuos: cada caso devuelve el inventario al estado en que lo encontró. Se ejecutan con una sola instrucción desde la raíz del proyecto, indicando contra qué nodo se quiere correr.

19.2. Resultados obtenidos

La Tabla 8 resume la ejecución registrada el 2 de noviembre de 2026 sobre el nodo 1, con ocho casos y treinta y seis comprobaciones individuales. Los ocho casos pasaron.

Tabla 8

Resumen de la ejecución de la suite de pruebas transaccionales

Caso	Nombre	Qué comprueba	Requisito	Estado	Tiempo
PT-01	Consulta de disponibilidad distribuida	La consulta llega a las tres bibliotecas y el resultado se presenta unificado por sede, sin que los disponibles superen el total de ejemplares.	RF-05, RNF-04	Pasa	2,09 s
PT-02	Sincronización entre bibliotecas	Una operación hecha en una sede se refleja de inmediato en la consulta agregada de las tres, sin reiniciar ni sincronizar a mano.	RF-05, RNF-08	Pasa	0,09 s

Caso	Nombre	Qué comprueba	Requisito	Estado	Tiempo
PT-03	Registro de préstamo	El préstamo descuenta exactamente un ejemplar, queda en estado activo y recibe una fecha de vencimiento posterior a la de entrega.	RF-06, RNF-02	Pasa	0,03 s
PT-04	Registro de devolución	La devolución cierra el préstamo y devuelve el ejemplar al inventario disponible.	RF-07, RNF-02	Pasa	0,01 s
PT-05	Reglas de negocio y códigos de error	Las operaciones inválidas se rechazan con el código correcto y el servidor sigue en pie.	RNF-02	Pasa	0,02 s
PT-06	Atomicidad de la transacción	Una operación rechazada no deja rastro: ni préstamo registrado ni ejemplar descontado.	RNF-02	Pasa	0,02 s
PT-07	Tolerancia a un nodo caído	Con una sede inalcanzable el sistema responde con las demás, informa cuál falló y no presenta datos del nodo caído.	RNF-05	Pasa	3,87 s
PT-08	Concurrencia sobre el mismo título	Varios préstamos simultáneos del mismo título no entregan más ejemplares de los que existen ni el mismo ejemplar dos veces.	RNF-02	Pasa	0,04 s

Nota. Ejecución del 02/11/2026 a las 22:22 sobre el nodo 1: ocho casos superados, ninguno fallido. El informe completo, con las treinta y seis comprobaciones y los valores observados, queda en el repositorio dentro de *docs/evidencias/*.

19.3. Los dos casos que sostienen el diseño

De los ocho casos hay dos que merecen explicación aparte, porque son los que responden a las preguntas difíciles sobre un sistema distribuido.

PT-02: cómo se mantiene la coherencia sin replicación

La objeción natural al diseño es que, si no hay replicación entre bibliotecas, los datos podrían quedar desalineados. El caso PT-02 muestra por qué no ocurre. Toma un título con un ejemplar libre, verifica que la consulta agregada lo ve, registra un préstamo en su sede, vuelve a consultar y comprueba que la vista global ya muestra cero disponibles; después registra la devolución y

comprueba que vuelve a mostrar uno. En la ejecución registrada el título «Designing Data-Intensive Applications» pasó de 1 a 0 y de vuelta a 1 disponibles.

El punto es que entre esos tres momentos no hubo ninguna sincronización. No la hubo porque no hace falta: la vista global no es una copia que haya que poner al día, sino una suma que se recalcula cada vez que alguien pregunta. Esta es la forma concreta en que el sistema mantiene la coherencia entre bibliotecas.

PT-08: concurrencia sobre el último ejemplar

Es el caso crítico del sistema transaccional. Comprueba que dos bibliotecarios que intentan prestar al mismo tiempo el último ejemplar de un título no lo consigan los dos. Para que la prueba sea real y no una simulación, el caso abre una conexión independiente por cada solicitud y las sincroniza con una barrera, de modo que todas lleguen a la operación de préstamo en el mismo instante en lugar de hacerlo una tras otra.

En la ejecución registrada se lanzaron dos solicitudes simultáneas sobre un título con un solo ejemplar: una fue aceptada, la otra fue rechazada por falta de disponibilidad, no se entregó ningún ejemplar dos veces y el inventario quedó en cero y volvió a uno tras la limpieza.

Un resultado así, por sí solo, no prueba gran cosa: una prueba de concurrencia mal construida pasa siempre, incluso sobre una implementación defectuosa. Para descartarlo se verificó la prueba contra una versión del servidor deliberadamente rota, sin el bloqueo de filas que protege la selección del ejemplar. Con esa versión el caso PT-08 falló como debía, reportando que se esperaba un ejemplar entregado y se habían entregado dos, y nombrando el ejemplar que se había entregado por duplicado. Solo después de comprobar que la prueba detecta el error se tomó como válido que lo pase la implementación real.

19.4. Qué queda sin cubrir

Conviene ser explícito sobre los límites de esta evidencia. La ejecución registrada se hizo con un solo nodo levantado, de modo que PT-01 y PT-07 comprobaron el comportamiento distribuido combinando ese nodo con sedes no disponibles; la ejecución con las tres bases activas queda

pendiente de registrar. El caso PT-05 no pudo comprobar el rechazo por falta de ejemplares, porque en ese momento no había ningún título agotado en el nodo de prueba. Y el requerimiento RNF-01, que es el de autenticación y control de roles, no tiene caso de prueba porque todavía no está implementado.

20. Gestión del proyecto

20.1. Enfoque de trabajo

El proyecto se desarrolla de forma incremental, en entregas sucesivas que coinciden con las fases de la asignatura. Cada entrega deja algo que funciona de punta a punta, en lugar de acumular partes que solo se integran al final. Esa decisión explica el orden en que se construyó el sistema: primero el esquema de base de datos y la capa de acceso a datos, después el servidor de nodo con su contrato, luego el cliente distribuido, después la interfaz, y por último la reorganización en capas y la suite de pruebas.

El trabajo se coordina sobre un repositorio compartido en GitHub, con la rama principal como línea estable y ramas de trabajo para los cambios que afectan a varios archivos a la vez, como ocurrió con la reorganización por capas. El avance queda registrado en el historial de confirmaciones, que es también la evidencia de la gestión.

20.2. Roles y responsabilidades

El equipo son tres estudiantes y el reparto de responsabilidades se refleja en los artefactos del proyecto, como muestra la Tabla 9.

Tabla 9

Roles del equipo y evidencia asociada

Integrante	Rol principal	Responsabilidades	Evidencia
Juan Felipe Villaquirán	Coordinación del repositorio	Administración del repositorio compartido, integración de los aportes y control de la rama principal.	Titular del repositorio en GitHub
Alejandro Mier Cadrazco	Desarrollo e integración	Servidor de nodo, contrato gRPC, capa de datos, confirmación y publicación de los cambios.	Historial de confirmaciones
Gabriel Esteban Badillo	Documentación y diseño	Documento maestro, requerimientos,	Autoría del documento

Integrante	Rol principal	Responsabilidades	Evidencia
		diagramas UML y modelo relacional.	

Nota. Elaboración propia. Los tres integrantes participan en el desarrollo; la tabla recoge la responsabilidad principal de cada uno, no una división exclusiva del trabajo.

20.3. Cronograma y hitos alcanzados

La Tabla 10 recoge los hitos del proyecto con la fecha en que quedaron registrados en el repositorio. No son fechas estimadas sino las del historial de confirmaciones, que es lo que permite verificarlas.

Tabla 10

Hitos del proyecto según el historial del repositorio

Fecha	Hito	Contenido de la entrega	Fase
25/09/2026	Puesta en marcha del repositorio y del servidor	Estructura inicial del proyecto, servidor de nodo en Python y script de creación de la base de datos.	Fase I
17/10/2026	Documento maestro	Planteamiento, marco conceptual, marco contextual, requerimientos, arquitectura, casos de uso, diagrama de clases y modelo relacional.	Fase II
18/10/2026	Primera versión de la interfaz	Pantallas de usuarios y préstamos sobre el cliente distribuido.	Fase II
23/10/2026	Reorganización en capas	Separación del código en presentación, negocio, entidad y datos, con la regla de dependencia en un solo sentido.	Fase III
02/11/2026	Suite de pruebas transaccionales	Ocho casos de prueba de extremo a extremo, informe de evidencias y actualización del manual del repositorio.	Fase III
05/11/2026	Interfaz adaptable y navegación	Menú de navegación, escalado según la pantalla, mensajes diferenciados de éxito y error, y corrección de los problemas de visualización.	Fase III

Nota. Elaboración propia a partir del historial del repositorio. La fecha del documento maestro corresponde a la entrega de la Fase II.

20.4. Gestión de la configuración

El repositorio contiene el código fuente, el script de creación de la base de datos, las herramientas de carga de datos y diagnóstico, la suite de pruebas y la documentación. Quedan

deliberadamente fuera del control de versiones tres cosas: el archivo de configuración local con los datos de conexión, el entorno virtual de Python y los archivos generados automáticamente a partir del contrato de Protocol Buffers.

La exclusión del archivo de configuración es una medida de seguridad y no una comodidad. Los datos de conexión a SQL Server, incluido el nombre del servidor y las credenciales cuando se usa autenticación propia del motor, se leen de variables de entorno y nunca se escriben en el código. Cada integrante mantiene su propio archivo local, porque el nombre de la instancia de SQL Server cambia de un equipo a otro.

20.5. Riesgos identificados

La Tabla 11 recoge los riesgos que el equipo ha identificado, con la medida prevista para cada uno. Están ordenados por el efecto que tendrían sobre la entrega.

Tabla 11

Riesgos del proyecto y medidas previstas

Riesgo	**Efecto**	**Medida prevista**
Las tres bases de datos no están disponibles el día de la sustentación, por depender de tres equipos distintos.	Alto	La aplicación detecta que ningún nodo responde y conmuta sola a un conjunto de datos de prueba en memoria, avisándolo en la barra inferior. La sustentación puede hacerse aunque falten los servidores.
El módulo de interpretación en lenguaje natural no alcanza a implementarse.	Alto	Tres alternativas con el mismo contrato: el modelo propio del apartado 18, el servicio externo y las

Riesgo	Efecto	Medida prevista
		reglas lingüísticas locales. Escoger una no obliga a cambiar el resto del sistema.
La autenticación y el control de roles del RNF-01 siguen sin implementarse.	Medio	Está declarada como brecha abierta en el apartado 12.3 y forma parte del plan de continuidad. No bloquea el resto de las operaciones.
El historial de confirmaciones concentra la autoría en un solo integrante, lo que dificulta evidenciar el aporte individual.	Medio	Repartir las confirmaciones de las siguientes entregas entre los tres integrantes, de modo que el historial refleje la participación real.
Las diferencias de configuración de SQL Server entre equipos impiden ejecutar el sistema.	Bajo	Herramienta de diagnóstico que prueba las combinaciones de controlador y servidor e indica qué valores poner en la configuración local.

Nota. Elaboración propia.

20.6. Estado actual y plan de continuidad

A la fecha de esta entrega están terminados el esquema de las tres bases de datos, la capa de datos con sus transacciones, las reglas de préstamo y devolución, el contrato gRPC, el servidor de nodo, el cliente distribuido, la aplicación de escritorio y la suite de pruebas transaccionales.

Quedan pendientes tres componentes, en este orden de prioridad. El primero es la autenticación con control de roles, que cierra el requerimiento RNF-01 y es la brecha más visible. El segundo es el intérprete de lenguaje natural descrito en el apartado 18, con la construcción del conjunto de datos como primer paso. El tercero son los reportes de inventario y mora del requerimiento RF-10, que hoy se obtienen consultando las pantallas pero no se exportan.

21. Conclusiones

1. El sistema distribuido permite integrar la información de tres bibliotecas independientes en una sola aplicación, facilitando la consulta de libros, su ubicación y disponibilidad sin necesidad de realizar búsquedas por separado en cada biblioteca.
2. La integración de gRPC/RPC, SQL Server e Inteligencia Artificial permite establecer una arquitectura que facilita la comunicación entre las bibliotecas y realizar consultas mediante lenguaje natural, mejorando la forma en que los usuarios interactúan con la información bibliográfica.
3. El uso de transacciones para los procesos de préstamos y devoluciones permite mantener la consistencia de la información, evitando que el estado de los ejemplares y los registros de préstamos queden desactualizados o presenten inconsistencias.
4. Definir diez requerimientos funcionales con su resultado esperado y nueve no funcionales con su criterio de aceptación deja claro qué tiene que hacer el sistema y bajo qué condiciones debe operar. El desarrollo individual de cada requerimiento no funcional en el apartado 12.3, con la decisión de diseño que lo sostiene y la prueba que lo comprueba, convierte esa lista en algo verificable y no en una declaración de intenciones.
5. Limitar el servicio de Inteligencia Artificial a interpretar la pregunta y redactar la respuesta, sin dejarlo calcular disponibilidad, asegura que toda cifra que vea el usuario venga de las bases de datos de los nodos. Esa decisión es la que mantiene la confiabilidad del sistema transaccional.
6. La revisión del estado del arte confirma que las dos aproximaciones existentes resuelven la mitad del problema cada una: los sistemas integrados de gestión bibliotecaria operan con garantías transaccionales pero exigen una base de datos compartida, y los protocolos de búsqueda federada respetan la autonomía de cada institución pero solo permiten consultar. La combinación que propone el prototipo,

autonomía total con operación transaccional en el nodo dueño del ejemplar, es la que responde a la restricción de la que parte el proyecto.

7. La suite de pruebas transaccionales aporta la evidencia que faltaba sobre el comportamiento del prototipo: ocho casos y treinta y seis comprobaciones que cubren disponibilidad distribuida, préstamo, devolución, atomicidad, códigos de error, caída de un nodo y concurrencia sobre el último ejemplar. Haber verificado la prueba de concurrencia contra una implementación deliberadamente defectuosa, y comprobar que en ese caso falla, es lo que permite tomar en serio que la pase la implementación real.
8. Prescindir de la replicación entre bibliotecas, que en principio parecía una limitación, resultó ser la decisión que simplifica el conjunto. Como la disponibilidad global se recalcula en cada consulta en lugar de mantenerse copiada, no existen datos que puedan quedar desalineados, ninguna transacción necesita abarcar dos bases de datos y un nodo que vuelve de una caída no tiene nada que poner al día.
9. El diseño de la red neuronal para interpretar las consultas aporta algo que la búsqueda por coincidencia de texto no puede dar: una medida de confianza sobre su propia interpretación. Esa medida es la que permite cumplir el requerimiento RNF-03, porque habilita al sistema para reconocer que no entendió una pregunta y pedir precisión, en lugar de devolver siempre el resultado más parecido.

22. Referencias Bibliográficas

Affolter, K., Stockinger, K., y Bernstein, A. (2019). A comparative survey of recent natural language interfaces for databases. *The VLDB Journal, 28*(5), 793–819. https://doi.org/10.1007/s00778-019-00567-8

Bass, L., Clements, P., y Kazman, R. (2021). *Software architecture in practice* (4.ª ed.). Addison-Wesley Professional.

Cañete, J., Chaperon, G., Fuentes, R., Ho, J.-H., Kang, H., y Pérez, J. (2023). *Spanish pre-trained BERT model and evaluation data*. arXiv. https://arxiv.org/abs/2308.02976

Chen, Q., Zhuo, Z., y Wang, W. (2019). *BERT for joint intent classification and slot filling*. arXiv. https://arxiv.org/abs/1902.10909

China, C. R. (2026, 27 de mayo). *¿Qué son los sistemas distribuidos?* IBM. https://www.ibm.com/es-es/think/topics/distributed-systems

Devlin, J., Chang, M.-W., Lee, K., y Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. En *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics* (pp. 4171–4186). Association for Computational Linguistics.

gRPC. (2024a, 12 de noviembre). *Introduction to gRPC*. https://grpc.io/docs/what-is-grpc/introduction/

gRPC. (2024b). *Core concepts, architecture and lifecycle*. https://grpc.io/docs/guides/concepts/

gRPC. (2024c, 25 de noviembre). *Quick start: Python*. https://grpc.io/docs/languages/python/quickstart/

gRPC. (2024d). *Basics tutorial: Python*. https://grpc.io/docs/languages/python/basics/

Härder, T., y Reuter, A. (1983). Principles of transaction-oriented database recovery. *ACM Computing Surveys, 15*(4), 287–317. https://doi.org/10.1145/289.291

Koha Community. (s.f.). *About Koha*. Recuperado el 5 de noviembre de 2026, de https://koha-community.org/about/

Microsoft. (2026a). *SQL Database Engine*. Microsoft Learn. https://learn.microsoft.com/en-us/sql/database-engine/sql-database-engine

Microsoft. (2026b). *SET TRANSACTION ISOLATION LEVEL (Transact-SQL)*. Microsoft Learn. https://learn.microsoft.com/en-us/sql/t-sql/statements/set-transaction-isolation-level-transact-sql

Microsoft. (2026c). *Iniciar transacción distribuida (Transact-SQL)*. Microsoft Learn. https://learn.microsoft.com/es-es/sql/t-sql/language-elements/begin-distributed-transaction-transact-sql

Organización Internacional de Normalización. (2023). *ISO/IEC 25010:2023. Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*. ISO.

Organización Internacional de Normalización, Comisión Electrotécnica Internacional e Instituto de Ingenieros Eléctricos y Electrónicos. (2018). *ISO/IEC/IEEE 29148:2018. Systems and software engineering — Life cycle processes — Requirements engineering*. ISO.

Python Software Foundation. (2026a). *Python 3.14.7 documentation*. Python.org. https://docs.python.org/3/

Python Software Foundation. (2026b). *sqlite3 — DB-API 2.0 interface for SQLite databases*. Python Documentation. https://docs.python.org/3/library/sqlite3.html

Russell, S. J., y Norvig, P. (2021). *Artificial intelligence: A modern approach* (4.ª ed.). Pearson.

van Steen, M. R., y Tanenbaum, A. S. (2017). *Distributed systems* (3.ª ed.). https://www.distributed-systems.net/index.php/books/distributed-systems-3rd-edition-2017/

Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., Ma, J., Li, I., Yao, Q., Roman, S., Zhang, Z., y Radev, D. (2018). Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL. En *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing* (pp. 3911–3921). Association for Computational Linguistics.